



Brock University
Faculty of Mathematics & Science
Department of Computer Science

COSC 3P93: Parallel Computing
4P78 Assignment 2

Prepared By:

Parker TenBroeck 7376726
pt21zs@brocku.ca

Instructor:
Robson De Grande

December 10, 2025

1 Performance

1.1

1. Define Task Latency and determine the Average Task Latency for the entire batch of 10 tasks
how long it takes for a system to start or complete a task after the task is issued.

$$\frac{0 + 0 + 0 + 0 + 1 + 2 + 2.5 + 3 + 3.5 + 5}{10} = 1.7s$$

2. Define Task Throughput and determine the Overall Task Throughput in the observed time interval

The number of tasks the system can complete in a given duration of time.

$$\frac{10}{7} = 1.43s$$

2 Parallel Design Patterns

2.1

1. I3 : $A[10] = X + 10$ (T1 write) $\rightarrow$ I4 : $Y = A[10] + 5$ (T2 read)
True (Flow) a write to $A[10]$ is later read by the other thread, so the flow of data is from I3 to I4
2. I1 : $P = P + 1$ (T1 write) and I2 : $P = P + 2$ (T2 write)
Output (Write-Write) both instructions write the same scalar P

I3 : $X = A[5]$ (T1 read) $\rightarrow$ I4 : $A[5] = 2 \times Y$ (T2 write)
(Read $\rightarrow$ Write) — I3 reads $A[5]$ and a later instruction writes $A[5]$, so the read precedes the write (anti).
3. I3 : $M[0][1] = 1$ (T1 write) and I4 : $M[0][1] = 2$ (T2 write)
Output (Write-Write) — both write the same matrix element, so there is an output dependency.
4. No Dependency's for any instructions
5. I3 : $A[8] = Y$ (T1 write) $\rightarrow$ I4 : $Y = A[8] + 1$ (T2 read of $A[8]$)
Type: True (Flow) — I3 writes $A[8]$ and I4 reads that location.

I3 : $A[8] = Y$ (T1 reads Y) and I4 : $Y = A[8] + 1$ (T2 writes Y)
Anti-Dependency (Read $\rightarrow$ Write) on Y — I3 reads Y while I4 later writes Y.

2.2

- 1.

2.3

- 1.

2.4

- 1.

3 OpenMPI

3.1

1. [6] MPI_Init: starts the MPI environment
2. [7] MPI_Comm_rank: gives each process its rank (ID) in MPI_COMM_WORLD
3. [8] MPI_Comm_size: gives the total number of processes
4. [16] MPI_Sendrecv: Sends left_send (1 double) to left_neighbor with tag 0.
Simultaneously receives right_recv (1 double) from right_neighbor with tag 0.
Sendrecv is useful in situations where two processes both want to exchange data but it is not clear who should go first.
5. [20] MPI_Sendrecv: Sends right_send to right_neighbor with tag 1.
Simultaneously Receives left_recv from left_neighbor with tag 1.
6. [27] MPI_Barrier: is a global synchronization, all processes must reach the boundary before any can proceed past it
7. [31] MPI_Finalize: Cleans up the MPI environment

3.2

1. [9] MPI_Init: starts the MPI environment
2. [10] MPI_Comm_rank: gives each process its rank (ID) in MPI_COMM_WORLD
3. [11] MPI_Comm_size: gives the total number of processes
4. [16] MPI_Scatter: distribute the dataset by splitting global_data into size blocks of length local_N and sends a block to each process
5. [21] MPI_Bcast: Broadcast from rank 0 to all other processes so each process has a copy of the centroid array.
6. [25] MPI_Reduce: each process gives its local_sum and the reduce operation sums all the values together.
7. [32] MPI_Gather: collect all local means on rank 0
8. [36] MPI_Allreduce: Reduce all values with MAX operation (so find the max of all values) and share it to all ranks
9. [40] MPI_Allgather: Collect all the sums from every process into the array.
10. [44] MPI_Finalize: Cleans up the MPI environment

4

5 Tools

5.1

1. Factors for sequential performance include, time complexity, quality of implementation, CPU speed, ILP, Memory behavior(cache, memory bandwidth), Compiler optimizations, Instruction throughput
2. Factors for parallel performance include, Parallel fraction of the algorithm, load balancing, synchronization overhead, communication overhead, memory contention, scalability, thread management overhead, cache coherence overhead.

6 Parallel Program Design

6.1

The methodological design of parallel programs is a structured approach that transforms a problem into an efficient parallel solution by identifying parallel work, organizing tasks, and minimizing communication and synchronization so the program scales and performs well on multiple processors

1. Partitioning
2. Communication
3. Agglomeration
4. Mapping
5. Synchronization
6. Performance Tuning